

W Y K Ł A D 5

Moduły, pakiety i wyjątki

import, pip, try/except, raise, with

Wprowadzenie do programowania II

Informatyka Stosowana | semestr letni 2025/2026

Akademia Nauk Stosowanych w Nowym Sączu

Tematy

- 1 Moduły - import, sys.path, __name__, dir()
- 2 Pakiety - __init__.py, struktura, import względny
- 3 pip i środowiska — instalacja, requirements.txt, venv
- 4 Wyjątki - try/except/else/finally, raise
- 5 Hierarchia wyjątków i własne klasy wyjątków
- 6 Menedżer kontekstu - instrukcja with
- 7 Testowanie - pytest, assert, struktura projektu
- 8 Dobre praktyki i organizacja kodu

Module

```
import, sys.path, __name__, __main__
```

Czym jest moduł?

Moduł to po prostu plik .py - podział kodu na pliki poprawia czytelność i umożliwia wielokrotne użycie.

Trzy rodzaje modułów

- ✓ **Napisany w Pythonie** dowolny plik .py
- ✓ **Napisany w C** np. moduł re (wyrażenia regularne)
- ✓ **Wbudowany (built-in)** np. itertools, sys - część interpretera

Przykład: math.py

```
# plik: pomocnik.py
def powitaj(imie):
    return f"Cześć, {imie}!"

PI = 3.14159
```

Dlaczego moduły?

- Prostota** Każdy moduł ma własny, ograniczony zakres odpowiedzialności
- Wielokrotne użycie** Import w dowolnym innym pliku bez kopiowania kodu
- Przestrzeń nazw** Unikanie kolizji nazw - math.pi vs numpy.pi
- Utrzymanie** Zmiany w jednym pliku nie psują reszty projektu

Instrukcja import - warianty

Kilka sposobów importowania — każdy z innymi konsekwencjami dla przestrzeni nazw.

```
# 1. Cały moduł
import math
print(math.pi)          # 3.14159...

# 2. Konkretne nazwy
from math import pi, sqrt
print(pi)                # bez przedrostka

# 3. Alias
import numpy as np
np.array([1, 2, 3])

# 4. Alias dla nazwy
from datetime import datetime as dt
dt.now()
```

X Unikaj from X import *

```
from math import * # RYZYKOWNE!
# importuje WSZYSTKO do namespace
# może nadpisać istniejące nazwy
# trudno znaleźć źródło nazwy
```

Import wewnątrz funkcji

```
def oblicz():
    from timeit import timeit # OK!
    start = timeit()
    # ... obliczenia ...
    return timeit() - start
# Moduł ładowany tylko gdy potrzebny
```

sys.path - gdzie Python szuka modułów

Python przeszukuje listę katalogów w określonej kolejności.

Kolejność przeszukiwania

- 1 Katalog bieżącego skryptu**
lub katalog roboczy (interaktywnie)
- 2 PYTHONPATH**
zmienna środowiskowa (jeśli ustawiona)
- 3 Katalogi instalacji**
site-packages, biblioteka standardowa

Podgląd i modyfikacja

```
import sys
print(sys.path)
# ['', '/usr/lib/...',
#   '/usr/lib/python3/...']

# Dodanie katalogu:
sys.path.append(
    r"C:\moje\libs"
)

# Lokalizacja modułu:
import os
print(os.__file__)
```

```
import sys; print("math" in sys.modules) # True/False - sprawdzenie czy moduł już załadowany
```

__name__ i __main__

Kluczowy mechanizm — rozróżnianie uruchomienia od importu.

Plik: test.py

```
print("nazwa:", __name__)

def main():
    print("Uruchamiam program!")

if __name__ == "__main__":
    main()
```

Wyniki

python test.py

```
nazwa: __main__
Uruchamiam program!
```

import test

```
nazwa: test
```

Zasada

Gdy uruchamiamy plik bezpośrednio: `__name__ == "__main__"`. Gdy importujemy: `__name__ == nazwa_modułu`.

Dzięki temu możemy pisać pliki, które działają zarówno jako programy, jak i jako moduły do importu. To standardowy wzorzec w Pythonie.

dir(), __all__ i introspekcja

Narzędzia do eksploracji zawartości modułów i kontroli eksportu.

dir() - lista nazw w module

```
import math
dir(math) # ['acos', 'asin', ...]

# Bez argumentu – bieżąca przestrzeń:
dir()     # ['__builtins__', ...]
```

importlib.reload()

```
import importlib
import moj_modul

# Po zmianie pliku:
importlib.reload(moj_modul)
```

__all__ - kontrola from X import *

```
# plik: helpers.py
__all__ = ["powitaj", "pozegnaj"]

def powitaj(): ...
def pozegnaj(): ...
def _prywatna(): ... # NIE w __all__
```



Buforowanie modułów

Moduł jest ładowany tylko raz - przy pierwszym imporcie. Kolejne import nie wykonują ponownie kodu modułu. **sys.modules** przechowuje słownik wszystkich załadowanych modułów. Użyj `importlib.reload()` aby wymusić ponowne załadowanie.

Biblioteka standardowa

Python zawiera bogatą bibliotekę standardową — nie trzeba niczego instalować.

os, pathlib	Operacje na plikach i katalogach
sys	Parametry interpretera, sys.path, sys.argv
json	Serializacja / deserializacja JSON
datetime	Daty, czasy, różnice czasowe
re	Wyrażenia regularne
collections	Counter, defaultdict, namedtuple, deque
itertools	Permutacje, kombinacje, łańcuchy iteratorów
functools	lru_cache, partial, reduce, wraps
math, random	Funkcje matematyczne, liczby losowe
unittest, doctest	Testowanie kodu
typing	Type hints: List, Dict, Optional, Union
logging	System logowania zamiast print()

Pakiety

`__init__.py`, struktura projektów, import względny

Pakiet - katalog z modułami

Pakiet to katalog z plikami .py. Plik `__init__.py` czyni go „zwykłym pakietem”.

Struktura pakietu

```
pkg/  
├── __init__.py  
├── md1.py  
├── md2.py  
└── md3.py  
  
# md1.py  
def fun1():  
    print("fun1")  
class K1:  
    pass
```

Import z pakietu

```
# Cały moduł z pakietu  
import pkg.md1  
pkg.md1.fun1()  
  
# Konkretna nazwa  
from pkg.md1 import fun1  
fun1() # bezpośrednio  
  
# Alias  
from pkg.md2 import K2 as Klasa2
```

Uwaga: `import pkg` sam w sobie nie importuje modułów pakietu! Potrzebny jest `__init__.py` z odpowiednią zawartością lub jawny import poszczególnych modułów.

__init__.py - inicjalizacja pakietu

Wykonywany przy pierwszym imporcie pakietu. Może być pusty lub zawierać logikę.

Przykładowy __init__.py

```
# pkg/__init__.py
print("Importowanie pkg")

L = [1, 2, 3] # obiekt dostępny

import pkg.md1
from pkg.md2 import fun2
from pkg.md3 import K3
```

__all__ w __init__.py

```
# pkg/__init__.py
__all__ = ["md2", "md3"]

# Teraz: from pkg import *
# zaimportuje TYLKO md2 i md3

# Bez __all__:
# from pkg import * → nic!
```



Nowoczesne podejście

Od Pythona 3.3 __init__.py nie jest obowiązkowy — katalog bez niego to "namespace package" (PEP 420).
W praktyce jednak warto go mieć: daje pełną kontrolę nad inicjalizacją, eksportem i zachowaniem pakietu.

Pakiety zagnieżdżone i import względny

Pakiety mogą zawierać subpakiety. Import względny używa kropek.

Struktura zagnieżdżona

```
pkg/  
├── __init__.py  
├── pkg1/  
│   ├── __init__.py  
│   ├── md1.py  
│   └── md2.py  
└── pkg2/  
    ├── __init__.py  
    └── md3.py
```

Import względny (z kropkami)

```
# W pliku pkg/pkg2/md3.py:  
  
# Import z nadrzędnego pakietu  
from .. import pkg1  
  
# Import z sąsiedniego subpakietu  
from ..pkg1.md2 import fun2  
  
# . = bieżący pakiet  
# .. = pakiet nadrzędny
```

Trzy typy obiektów tworzonych przez import:

moduł

plik .py

zwykły pakiet

katalog z __init__.py

namespace package

katalog bez __init__.py (od 3.3)

pip i środowiska wirtualne

Instalacja pakietów, PyPI, venv

pip - menedżer pakietów

pip pobiera pakiety z PyPI (Python Package Index) - ponad 500 000 projektów.

Najważniejsze komendy pip

<code>pip install pygame</code>	Instalacja najnowszej wersji
<code>pip install pygame==2.6.0</code>	Konkretna wersja
<code>pip install --upgrade pygame</code>	Aktualizacja do najnowszej
<code>pip uninstall pygame</code>	Usunięcie pakietu
<code>pip show pygame</code>	Informacje o pakiecie (wersja, lokalizacja)
<code>pip list</code>	Lista zainstalowanych pakietów
<code>pip freeze > requirements.txt</code>	Eksport wersji do pliku
<code>pip install -r requirements.txt</code>	Instalacja z pliku wymagań
<code>pip index versions pygame</code>	Dostępne wersje na PyPI
<code>pip install --force-reinstall pkg</code>	Wymuszenie reinstalacji

requirements.txt i wersjonowanie

Plik wymagań umożliwia odtworzenie identycznego środowiska.

Przykładowy requirements.txt

```
# Dokładna wersja
numpy==1.26.4
pandas==2.2.3
matplotlib==3.9.2

# Minimalna wersja
requests>=2.28

# Zakres
Flask>=2.0,<3.0
```

Specyfikatory wersji

<code>==1.2.3</code>	Dokładnie ta wersja
<code>>=1.2</code>	Minimum 1.2
<code><=2.0</code>	Maksimum 2.0
<code>~=1.4.2</code>	≥1.4.2, <1.5.0 (kompatybilna)
<code>!=1.5.0</code>	Wyklucz konkretną wersję
<code>>=1.0,<2.0</code>	Zakres wersji

Workflow:

```
pip freeze > requirements.txt # eksport → git add requirements.txt → pip install -r requirements.txt # na innym komputerze
```


Środowiska wirtualne - venv

Izolacja zależności - każdy projekt może mieć inne wersje bibliotek.

Tworzenie i aktywacja

```
# Tworzenie środowiska
python -m venv .venv

# Aktywacja (Windows)
.venv\Scripts\activate

# Aktywacja (Linux/Mac)
source .venv/bin/activate

# Dezaktywacja
deactivate
```

Dlaczego venv?

- ✓ Każdy projekt ma własne zależności
- ✓ Brak konfliktów wersji między projektami
- ✓ Czysta lista w pip freeze
- ✓ Część biblioteki standardowej (nie trzeba instalować)
- ✗ NIE commitujemy .venv do gita!

💡 .gitignore

```
.venv/          # ZAWSZE w .gitignore!   Zamiast tego commitujemy: requirements.txt
```

Obsługa wyjątków

try, except, else, finally, raise

Czym są wyjątki?

Wyjątek to sygnał błędu - zamiast przerwać program, można go obsłużyć.

Bez obsługi - program pada

```
print(1 / 0)
# ZeroDivisionError:
#   divisio by zero
# → Program się zatrzymuje!
```

Z obsługą - program działa dalej

```
try:
    print(1 / 0)
except ZeroDivisionError:
    print("Nie dziel przez zero!")
print("Program działa dalej")
```

Najczęstsze wyjątki

ValueError	zła wartość	<code>int("abc")</code>	FileNotFoundError	brak pliku	<code>open("brak.txt")</code>
TypeError	zły typ	<code>"a" + 1</code>	AttributeError	brak atrybutu	<code>"abc".brak</code>
IndexError	indeks poza zakresem	<code>[1,2][5]</code>	NameError	niezdefiniowana nazwa	<code>print(xyz)</code>
KeyError	brak klucza w dict	<code>{"a":1}["b"]</code>	ZeroDivisionError	dzielenie przez zero	<code>1 / 0</code>

try / except - podstawy

Blok try zawiera kod ryzykowny, except obsługuje konkretny typ błędu.

Jeden typ wyjątku

```
try:
    wiek = int(input("Wiek: "))
    print(f"Za 10 lat: {wiek + 10}")
except ValueError:
    print("To nie jest liczba!")

# Dostęp do treści wyjątku:
except ValueError as e:
    print(f"Błąd: {e}")
```

Wiele typów wyjątków

```
try:
    dane = [1, 2, 3]
    idx = int(input("Indeks: "))
    print(dane[idx])
except ValueError:
    print("Podaj liczbę!")
except IndexError:
    print("Poza zakresem!")
except (TypeError, KeyError):
    print("Inny błąd danych")
```

Kolejność ma znaczenie:

except sprawdzany od góry. Jeśli złapiemy Exception przed ValueError, to ValueError nigdy nie zostanie obsłużony osobno.

else i finally

Pełna składnia: try / except / else / finally - każdy blok ma swoją rolę.

```
try:
    plik = open("dane.txt")
    dane = plik.read()
    liczba = int(dane)
except FileNotFoundError:
    print("Brak pliku!")
except ValueError:
    print("Zły format danych!")
else:
    # Wykona się TYLKO jeśli NIE było wyjątku
    print(f"Wczytano: {liczba}")
finally:
    # Wykona się ZAWSZE
    print("Sprzątanie...")
```

Przeptyw wykonania



raise - zgłaszanie wyjątków

Możemy sami zgłaszać wyjątki, gdy dane są niepoprawne.

raise z komunikatem

```
def ustaw_wiek(wiek):  
    if not isinstance(wiek, int):  
        raise TypeError("Wiek musi być int")  
    if not (0 <= wiek <= 150):  
        raise ValueError(  
            f"Wiek {wiek} poza zakresem"  
        )  
    return wiek
```

raise bez argumentu (re-raise)

```
try:  
    ryzykowna_operacja()  
except ValueError as e:  
    print(f"Loguję: {e}")  
    raise # ponownie zgłasza  
        # ten sam wyjątek  
  
# Przydatne: zaloguj i przekaz dalej
```

💡 raise ... from - łańcuch wyjątków (Python 3)

```
try:  
    v = int(dane)  
except ValueError as orig:  
    raise RuntimeError("Błąd przetwarzania") from orig # zachowuje oryginalny traceback
```

Hierarchia wyjątków

Wszystkie wyjątki dziedziczą z `BaseException`. Większość z `Exception`.

```
BaseException
├── SystemExit          # sys.exit()
├── KeyboardInterrupt  # Ctrl+C
├── GeneratorExit       # zamknięcie generatora
├── Exception           # ← BAZOWY dla większości
│   ├── ArithmeticError
│   │   ├── ZeroDivisionError
│   │   └── OverflowError
│   ├── LookupError
│   │   ├── IndexError
│   │   └── KeyError
│   ├── OSError
│   │   ├── FileNotFoundError
│   │   └── PermissionError
│   ├── ValueError
│   ├── TypeError
│   ├── AttributeError
│   ├── NameError
│   └── StopIteration  # koniec iteratora
```

Własne wyjątki - klasy

Definiujemy własną klasę dziedziczącą z Exception.

```
class WiekError(ValueError):
    """Błąd związany z wiekiem."""
    pass

class ZakresBledny(WiekError):
    def __init__(self, wiek, msg=None):
        self.wiek = wiek
        super().__init__(
            msg or f"Zły wiek: {wiek}"
        )

def sprawdz(wiek):
    if wiek < 0 or wiek > 150:
        raise ZakresBledny(wiek)
    return wiek
```

Użycie

```
try:
    sprawdz(-5)
except ZakresBledny as e:
    print(e)          # Zły wiek: -5
    print(e.wiek)     # -5
except WiekError:
    print("Inny błąd wieku")
```

Konwencje

- ✓ Nazwy kończą się na Error
- ✓ Dziedzicz z Exception (nie BaseException)
- ✓ Dodaj docstring
- ✓ Grupuj w hierarchię dla swojego projektu

Łapanie zbyt ogólnych wyjątków

except Exception lub gołe except: maskują prawdziwe błędy.

X Źle

```
try:
    dane = pobierz_dane()
    wynik = przetworz(dane)
except:
    pass # „po cichu” połykamy
        # WSZYSTKIE błędy!

# Równie złe:
except Exception:
    print("Coś poszło nie tak")
```

✓ Dobrze

```
try:
    dane = pobierz_dane()
    wynik = przetworz(dane)
except ConnectionError:
    print("Brak połączenia")
except ValueError as e:
    print(f"Złe dane: {e}")
except FileNotFoundError:
    print("Plik nie istnieje")
```

Dlaczego to ważne?

Gołe except: łapie nawet KeyboardInterrupt (Ctrl+C) i SystemExit! Zawsze łapmy konkretne typy wyjątków. Jeśli naprawdę trzeba złapać „wszystko”, używamy except Exception as e: i logujemy e.

Wyjątki w praktyce - walidacja

Przykład z notatek: konwersja liczb rzymskich z kontrolą błędów.

```
def arabskie_na_rzymskie(liczba):
    if not isinstance(liczba, int):
        raise TypeError("Wymagany typ int")
    if not (1 <= liczba <= 3999):
        raise ValueError(f"Liczba {liczba} poza zakresem 1-3999")

    wartosci = [(1000, "M"), (900, "CM"), (500, "D"), (400, "CD"),
                (100, "C"), (90, "XC"), (50, "L"), (40, "XL"),
                (10, "X"), (9, "IX"), (5, "V"), (4, "IV"), (1, "I")]
    wynik = ""
    for val, sym in wartosci:
        while liczba >= val:
            wynik += sym
            liczba -= val
    return wynik

# Użycie z obsługą wyjątków:
try:
    print(arabskie_na_rzymskie(1994)) # MCMXCIV
    print(arabskie_na_rzymskie(5000)) # ValueError!
except ValueError as e:
    print(f"Błąd: {e}")
```

Menedżer kontekstu

Instrukcja with i protokół kontekstu

Instrukcja with - bezpieczne zasoby

with gwarantuje zamknięcie zasobu nawet gdy wystąpi wyjątek.

X Bez with

```
plik = open("dane.txt")
try:
    dane = plik.read()
    # ... przetwarzanie ...
finally:
    plik.close() # MUSIMY pamiętać!
```

✓ Z with

```
with open("dane.txt") as plik:
    dane = plik.read()
    # ... przetwarzanie ...
# plik.close() wywołane AUTOMATYCZNIE
# nawet jeśli był wyjątek!
```

Wiele kontekstów naraz

```
with open("input.txt") as src, open("output.txt", "w") as dst:
    for line in src:
        dst.write(line.upper()) # oba pliki zamknięte automatycznie
```

Protokół kontekstu - `__enter__` / `__exit__`

Każdy obiekt może być użyty z `with`, jeśli implementuje dwie metody.

```
class Timer:
    def __enter__(self):
        import time
        self.start = time.perf_counter()
        return self    # → zmienna po "as"

    def __exit__(self, exc_type,
                 exc_val, exc_tb):
        import time
        elapsed = time.perf_counter()
                     - self.start
        print(f"Czas: {elapsed:.3f}s")
        return False   # nie tłumić wyjątków

with Timer() as t:
    sum(range(10_000_000))
# Czas: 0.187s
```

Jak to działa?

- 1 `__enter__()` wywołane na początku bloku `with`
- 2 Wartość zwracana trafia do zmiennej po `as`
- 3 Kod w bloku `with` się wykonuje
- 4 `__exit__()` wywołane na końcu (lub przy wyjątku)
- 5 Jeśli `__exit__` zwraca `True`, wyjątek jest tłumiony

EAFP vs LBYL - dwa style

Python preferuje EAFP: łatwiej prosić o przebaczenie niż o pozwolenie.

LBYL - Look Before You Leap

```
# Sprawdź PRZED operacją
if "klucz" in slownik:
    val = slownik["klucz"]
else:
    val = "domyślna"

if os.path.exists("plik.txt"):
    plik = open("plik.txt")
```

EAFP - Easier to Ask Forgiveness

```
# Próbuje i obsługuje wyjątek
try:
    val = slownik["klucz"]
except KeyError:
    val = "domyślna"

try:
    plik = open("plik.txt")
except FileNotFoundError:
    print("Brak pliku")
```

Dlaczego EAFP jest pythonowe?

LBYL ma problem z „wyścigiem” (race condition) - plik może zniknąć między sprawdzeniem a otwarciem.

EAFP jest też często szybsze - w typowym przypadku (brak błędu) nie ma kosztu dodatkowego sprawdzenia.

W Pythonie wyjątki są „tanie” w typowym przypadku, droższe tylko gdy faktycznie występują.

Testowanie

pytest, assert, struktura projektów

pytest - podstawy testowania

Framework do testów jednostkowych. Instalacja: `pip install pytest`

Plik: `test_kalkulator.py`

```
from src.kalkulator import dodaj

def test_dodaj_dwie_liczby():
    assert dodaj(2, 3) == 5

def test_dodaj_float():
    assert dodaj(2.5, 3.1) == 5.6

def test_dodaj_zero():
    assert dodaj(10, 0) == 10

def test_dodaj_negatywne():
    assert dodaj(-1, -1) == -2
```

Konwencje nazewnictwa

Pliki:	<code>test_*.py</code> lub <code>*_test.py</code>
Funkcje:	<code>test_*</code> () np. <code>test_dodaj()</code>
Klasy:	<code>Test*</code> np. <code>TestKalkulator</code>
Uruchomienie:	
Asercje:	<code>assert</code> wynik == oczekiwane

```
pytest -v          # szczegółowe
pytest -sv         # z printami
```

Testowanie wyjątków:

```
import pytest; def test_blad(): with pytest.raises(ValueError): int("abc") # oczekujemy ValueError
```


Struktura projektu z testami

Typowa organizacja kodu produkcyjnego i testów.

Struktura katalogów

```
projekt/  
├── src/  
│   ├── __init__.py  
│   ├── kalkulator.py  
│   └── walidator.py  
├── tests/  
│   ├── __init__.py  
│   ├── test_kalkulator.py  
│   └── test_walidator.py  
├── requirements.txt  
├── .gitignore  
└── README.md
```

src/kalkulator.py

```
def dodaj(a, b):  
    return a + b  
  
def podziel(a, b):  
    if b == 0:  
        raise ValueError("Dzielnik = 0")  
    return a / b
```

tests/test_kalkulator.py

```
import pytest  
from src.kalkulator import dodaj, podziel  
  
def test_dodaj():  
    assert dodaj(2, 3) == 5  
  
def test_podziel_przez_zero():  
    with pytest.raises(ValueError):  
        podziel(10, 0)
```

Dobre praktyki

Organizacja, import, wzorce

Dobre praktyki importów

PEP 8 definiuje kolejność i styl importów.

Kolejność importów (PEP 8)

```
# 1. Biblioteka standardowa
import os
import sys
from pathlib import Path

# 2. Biblioteki zewnętrzne (pip)
import numpy as np
import pandas as pd

# 3. Lokalne moduły projektu
from src.kalkulator import dodaj
from .helpers import format_dane

# Każda grupa oddzielona pustą linią!
```

Zasady

- ✓ Importy na początku pliku
- ✓ Jeden import na linię
- ✓ Aliasy dla długich nazw
- ✓ import X lepszy niż from X import *
- ✓ Grupowanie wg PEP 8
- ✗ Cykliczne importy — refaktoruj!
- ✗ from X import * w produkcji
- ✗ Import nieused modułów

Częste błędy z modułami i wyjątkami

Pięć najczęstszych pułapek.

Błąd	Rozwiązanie
Nazwy plików kolidują z modułami	Nie nazywaj pliku <code>math.py</code> , <code>random.py</code> itd.
Cykliczny import: <code>A→B→A</code>	Przesuń import do wnętrza funkcji lub refaktoruj
gołe <code>except</code> : (bez typu)	Łap konkretne typy: <code>except ValueError</code>
<code>pass</code> w <code>except</code> (połykanie błędów)	Loguj błąd: <code>logging.error(e)</code>
Brak <code>__init__.py</code> w pakiecie	Dodaj pusty <code>__init__.py</code> (lub użyj namespace pkg)

logging zamiast print()

Profesjonalny sposób rejestrowania zdarzeń w programie.

```
import logging

# Konfiguracja
logging.basicConfig(
    level=logging.DEBUG,
    format="%(asctime)s %(levelname)s "
           "%(message)s"
)
log = logging.getLogger(__name__)

def przetworz(dane):
    log.info(f"Przetwarzam {len(dane)} e1.")
    try:
        wynik = oblicz(dane)
        log.debug(f"Wynik: {wynik}")
        return wynik
    except ValueError as e:
        log.error(f"Błąd: {e}")
        raise
```

Poziomy logów

DEBUG	Szczegóły diagnostyczne
INFO	Potwierdzenie działania
WARNING	Coś nieoczekiwanego
ERROR	Poważny problem
CRITICAL	Program może się zatrzymać

Dlaczego nie print()?

Logi można filtrować po poziomie, kierować do pliku, wyłączać w produkcji. print() tego nie potrafi.

pathlib - nowoczesne operacje na ścieżkach

Moduł pathlib (od Pythona 3.4) zastępuje os.path obiektem API.

os.path (stary sposób)

```
import os

# Łączenie ścieżek
path = os.path.join("dane", "plik.txt")

# Sprawdzenie istnienia
os.path.exists(path)

# Rozszerzenie
os.path.splitext(path)[1] # .txt

# Odczyt
with open(path) as f:
    dane = f.read()
```

pathlib (nowoczesny sposób)

```
from pathlib import Path

# Łączenie ścieżek (operator /)
path = Path("dane") / "plik.txt"

# Sprawdzenie istnienia
path.exists()

# Rozszerzenie
path.suffix # .txt

# Odczyt (jedna linia!)
dane = path.read_text()

# Wyszukiwanie plików
for p in Path(".").glob("**/*.py"):
    print(p)
```

Moduł json - serializacja danych

Zamiana obiektów Pythona na format JSON i z powrotem.

Python → JSON (zapis)

```
import json

dane = {"imie": "Anna", "wiek": 25,
       "hobby": ["Python", "góry"]}

# Do pliku
with open("dane.json", "w") as f:
    json.dump(dane, f, indent=2)
```

JSON → Python (odczyt)

```
with open("dane.json") as f:
    wczytane = json.load(f)

print(wczytane["imie"]) # Anna

# Do/z stringa:
tekst = json.dumps(dane)
obiekt = json.loads(tekst)
```

Mapowanie typów:

dict ↔ object {}

list ↔ array []

str ↔ string

int/float ↔ number

True/False ↔ true/false

None ↔ null

Moduł collections - specjalne kontenery

Rozszerzenia typowych struktur danych — Counter, defaultdict, namedtuple.

Counter

```
from collections import Counter
txt = "abracadabra"
c = Counter(txt)
c.most_common(3)
# [('a',5),('b',2),('r',2)]
```

defaultdict

```
from collections \
    import defaultdict
d = defaultdict(list)
d["a"].append(1)
d["a"].append(2)
# {"a": [1, 2]}
```

namedtuple

```
from collections \
    import namedtuple
Punkt = namedtuple(
    "Punkt", ["x","y"])
p = Punkt(3, 4)
print(p.x) # 3
```

deque — kolejka dwustronna ($O(1)$ z obu stron)

```
from collections import deque
d = deque([1, 2, 3])
d.appendleft(0)      # [0, 1, 2, 3]     $O(1)$  – lista ma  $O(n)$ !
d.popleft()          # 0               $O(1)$  – lista ma  $O(n)$ !
d.rotate(1)          # [3, 0, 1, 2]   obrót w prawo
d = deque(maxlen=3)  # bufor cykliczny: po dodaniu 4. elementu, najstarszy wypada
```


Operator := (walrus) a wyjątki

Operator przypisania w wyrażeniu — przydatny w walidacji i pętlach.

Walidacja danych wejściowych

```
# Bez walrus:
linia = input()
while linia != "quit":
    print(linia.upper())
    linia = input()

# Z walrus (:=)
while (linia := input()) != "quit":
    print(linia.upper())
```

Z wyrażeniami regularnymi

```
import re
tekst = "Cena: 49.99 PLN"

if (m := re.search(r"\d+\.\d+",
                  tekst)):
    print(f"Znaleziono: {m.group()}")
    # 49.99
else:
    print("Brak dopasowania")
```

💡 Kiedy używać?

:= (PEP 572, Python 3.8) eliminuje powtórzone obliczenia i przypisania. Używać gdy oszczędza wyrażenie.

Nie nadużywać! Zbyt złożone wyrażenia z := stają się nieczytelne. Prostota przede wszystkim.

Ściga: składnia i wzorce

Najważniejsze konstrukcje z tego wykładu w jednym miejscu.

<code>import math</code>	Import całego modułu
<code>from math import pi</code>	Import konkretnej nazwy
<code>import numpy as np</code>	Import z aliasem
<code>if __name__ == "__main__":</code>	Guard — uruchomienie vs import
<code>sys.path.append("...")</code>	Dodanie ścieżki wyszukiwania
<code>pip install -r req.txt</code>	Instalacja z pliku wymagań
<code>python -m venv .venv</code>	Utworzenie środowiska wirtualnego
<code>try: / except E:</code>	Obsługa wyjątku typu E
<code>except E as e:</code>	Dostęp do obiektu wyjątku
<code>else: (po except)</code>	Kod gdy NIE było wyjątku
<code>finally:</code>	Kod ZAWSZE wykonywany
<code>raise ValueError("...")</code>	Zgłoszenie wyjątku
<code>with open() as f:</code>	Menedżer kontekstu
<code>pytest.raises(ValueError)</code>	Test czy wyjątek występuje

Import pod maską - co naprawdę robi Python

Mechanizm importu krok po kroku.

1

Sprawdza `sys.modules`

Czy moduł jest już w cache? Jeśli tak — używa go (bez ponownego wykonania kodu).

2

Szuka pliku na `sys.path`

Przeszukuje katalogi w kolejności: bieżący, `PYTHONPATH`, `site-packages`.

3

Tworzy obiekt modułu

Tworzy pusty obiekt `types.ModuleType` i dodaje go do `sys.modules`.

4

Wykonuje kod modułu

Uruchamia cały plik `.py` od początku do końca w kontekście nowego modułu.

5

Wiąże nazwę w callerze

`import math` → dodaje nazwę `'math'` do bieżącej przestrzeni nazw.

6

Tworzy `__pycache__/* .pyc`

Kompiluje bytecode do pliku `.pyc` dla szybszego ładowania następnym razem.

match/case - structural pattern matching

Od Pythona 3.10 (PEP 634) - elegancka alternatywa dla łańcuchów if/elif.

Podstawowe dopasowanie

```
def http_status(code):  
    match code:  
        case 200:  
            return "OK"  
        case 404:  
            return "Not Found"  
        case 500:  
            return "Server Error"  
        case _: # wildcard  
            return f"Kod: {code}"
```

Dopasowanie z or i typami

```
match command:  
    case "quit" | "exit" | "q":  
        sys.exit()  
    case str(s) if len(s) > 100:  
        print("Za długie!")  
    case [x, y]:  
        print(f"Para: {x}, {y}")  
    case {"action": action}:  
        wykonaj(action)  
    case _:  
        print("Nieznana komenda")
```

💡 `_` to wildcard (jak default w switch/case). Pola mogą być różnych typów: wartości, wzorce list, słowników, z guard clause (if).

Triki z wyjątkami

Zaawansowane wzorce obsługi błędów.

ExceptionGroup (Python 3.11+)

```
# Wiele wyjątków naraz!
try:
    raise ExceptionGroup("błędy", [
        ValueError("zła wartość"),
        TypeError("zły typ")
    ])
except* ValueError as eg:
    print(eg.exceptions)
```

traceback — szczegóły błędu

```
import traceback
try:
    1 / 0
except:
    traceback.print_exc()
    # pełny stack trace
    tb = traceback.format_exc()
    # jako string do logowania
```

contextlib.suppress — eleganckie pomijanie wyjątków

```
from contextlib import suppress

# Zamiast: try: os.remove("tmp.txt") except FileNotFoundError: pass
with suppress(FileNotFoundError):
    os.remove("tmp.txt") # Cisza jeśli plik nie istnieje, wyjątek jeśli inny błąd
```

Od zera do projektu - workflow

Typowy przebieg pracy z modułami, pakietami i wyjątkami w jednym projekcie.

```
# 1. Tworzymy środowisko
python -m venv .venv && source .venv/bin/activate

# 2. Instalujemy zależności
pip install requests pytest && pip freeze > requirements.txt

# 3. Piszemy moduł z obsługą błędów
# src/api.py
import requests
def pobierz(url):
    try:
        resp = requests.get(url, timeout=5)
        resp.raise_for_status()          # HTTPError jeśli status >= 400
        return resp.json()
    except requests.ConnectionError:
        raise ConnectionError(f"Brak połączenia z {url}")
    except requests.Timeout:
        raise TimeoutError(f"Timeout: {url}")

# 4. Piszemy testy
# tests/test_api.py
def test_pobierz_zly_url():
    with pytest.raises(ConnectionError):
        pobierz("http://nieistniejacy.serwer")
```

Moduły, pakiety i wyjątki

- | | | |
|----|--|--|
| 1 | Moduł = plik .py | <code>import, from...import, aliasy, __all__</code> |
| 2 | Pakiet = katalog | <code>__init__.py</code> , import względny, namespace packages |
| 3 | <code>if __name__ == "__main__":</code> | Rozróżnienie uruchomienia od importu |
| 4 | <code>pip + venv</code> | Izolacja zależności, <code>requirements.txt</code> |
| 5 | <code>try/except/else/finally</code> | Pełny blok obsługi wyjątków |
| 6 | <code>raise + własne klasy Error</code> | Zgłaszanie i definiowanie wyjątków |
| 7 | <code>with</code> — menedżer kontekstu | Bezpieczne zarządzanie zasobami |
| 8 | łap konkretne wyjątki! | Nigdy gołe <code>except</code> : ani <code>except Exception</code> |
| 9 | EAFP > LBYL | Pythonowy styl: próbuj i obsłuż |
| 10 | <code>pytest</code> | Testuj z <code>assert</code> i <code>pytest.raises</code> |

Następny wykład: Klasy i programowanie obiektowe